



Archivo central de inicialización. Carga todos los componentes HTML y llama sus funciones init cuando el DOM ya está listo.

Línea / Bloque	Explicación
<code>const container = document.getElementById()</code>	Busca el div contenedor antes de hacer fetch. Si no existe en esta página (ej: contacto no tiene carrito), sale sin error en lugar de romper.
<code>if (!container) return</code>	Guarda clave: evita que el fetch falle silenciosamente en páginas que no tienen ese contenedor.
<code>.catch(err => console.error())</code>	Se descomentó el catch para ver errores reales en consola durante el desarrollo.
Un solo DOMContentLoaded	El archivo tenía DOS bloques DOMContentLoaded. El segundo sobrescribía al primero. Se unificaron en uno solo.
<code>loadComponent("carrito-container", "cart.html", initCart)</code>	Línea nueva. Carga el aside del carrito como componente y llama a initCart() solo cuando el HTML ya está inyectado en el DOM.
<code>if (form)</code>	El listener del formulario de contacto se envolvió en un if(form) para que no rompa en páginas sin formulario (ej: catálogo).

4. cart.js — Explicación línea a línea

Contiene toda la lógica del carrito: agregar, actualizar, eliminar y mostrar productos. Vive dentro de `initCart()` para garantizar que el aside ya existe en el DOM.

Línea / Bloque	Explicación
<code>function initCart()</code>	Todo el código del carrito vive dentro de esta función. <code>main.js</code> la llama como callback de <code>loadComponent</code> , garantizando que el aside ya existe en el DOM.
<code>const carritoLateral = document.querySelector()</code>	Las referencias al DOM se toman DENTRO de <code>initCart</code> , no al cargar el script. Así los elementos ya existen cuando se buscan.
<code>carritoItems.innerHTML = ""</code>	Limpia el item de ejemplo hardcoded en <code>cart.html</code> para que el carrito arranque vacío.
<code>window.toggleCarrito = function()</code>	Se expone en <code>window</code> (scope global) para que los atributos <code>onclick="toggleCarrito()"</code> del HTML del carrito puedan encontrarla aunque viva dentro de <code>initCart</code> .
<code>function adjuntarBotonesCarrito()</code>	Función separada que recorre todos los <code>.btn-cart</code> y les pone el listener. Se puede llamar varias veces sin duplicar listeners.
<code>if (boton.dataset.cartReady) return</code> <code>boton.dataset.cartReady = "true"</code>	Flag anti-duplicado. Si un botón ya tiene listener (<code>cartReady="true"</code>), se salta. Evita que al llamar la función dos veces se ejecute doble el agregar.
<code>adjuntarBotonesCarrito()</code> primera llamada	Se llama al inicio de <code>initCart</code> para las tarjetas que ya están en el HTML estático del catálogo.
<code>document.addEventListener("catalogoListo", ...)</code>	Escucha el evento personalizado que lanza <code>catalogo.js</code> cuando termina de pintar tarjetas del <code>localStorage</code> . Llama <code>adjuntarBotonesCarrito()</code> para esos nuevos botones.
<code>carritoItems.querySelector([data-nombre="..."])</code>	Busca si el producto ya existe en el carrito por su nombre. Si lo encuentra, solo suma 1 en lugar de crear una fila nueva.
<code>div.dataset.nombre</code> <code>div.dataset.precio</code> <code>div.dataset.cantidad</code>	Guarda los datos del producto en atributos <code>data-*</code> del elemento DOM. Así <code>eliminarItem</code> y <code>actualizarCantidadItem</code> siempre leen valores actualizados.
<code>btn-restar-item</code> <code>btn-sumar-item</code>	Botones +/- generados dentro del carrito (no en la tarjeta del catálogo). Cada uno lee y escribe <code>div.dataset.cantidad</code> para mantener el estado.
<code>if (nueva < 1) return</code>	El restar tiene piso en 1. Para eliminar completamente hay que usar el botón

```
contenedor.innerHTML +=  
cardHTML
```

```
document.dispatchEvent( new  
CustomEvent("catalogoListo"))
```

Agrega cada tarjeta del localStorage al grid. Se usa += para acumular sin borrar las tarjetas hardcodeadas en el HTML estático.

Al terminar de pintar todas las tarjetas dinámicas, lanza un evento personalizado. cart.js lo escucha y adjunta los listeners del carrito a esos nuevos botones.

6. cart.html — Explicación línea a línea

Estructura del aside lateral. Se limpió para que no tenga datos hardcodeados y se vinculó el CSS del componente.

Línea / Bloque	Explicación
<pre><link rel="stylesheet" href="/components/cart/cart.css"></pre>	Línea nueva. Vincula el CSS del carrito directamente en el componente para que cargue junto con el HTML del aside.
<pre><div id="carrito-items"> <!-- vacío --></pre>	Se eliminó el div.carrito-item de ejemplo que estaba hardcodeado. El body del carrito arranca vacío; cart.js llena con divs reales.
<pre><p class="conteo-productos"> 0 productos</pre>	Cambiado de "2 productos" a "0 productos" para que el estado inicial sea correcto antes de que el JS tome el control.
<pre><strong id="subtotal-valor"> \$0</pre>	Valor inicial corregido a \$0 en lugar de \$0.00 para consistencia con el formato es-CO que usa el JS.

7. cart.css — Explicación línea a línea

Archivo creado desde cero. Implementa el panel lateral deslizante y la grilla de 4 columnas usando las variables de `paletaColores.css`.

Línea / Bloque	Explicación
<code>.carrito-overlay display: none</code>	Oculto por defecto. Al agregar clase <code>.activo</code> se muestra cubriendo toda la pantalla con <code>rgba(0,0,0,0.45)</code> para oscurecer el fondo detrás del aside.
<code>.carrito-sidebar position: fixed right: -480px</code>	Panel oculto fuera de pantalla hacia la derecha. Esta es la clave del panel lateral: no ocupa espacio en el flujo del documento y espera fuera del viewport.
<code>.carrito-sidebar.abierto right: 0</code>	Cuando JS agrega la clase <code>.abierto</code> , el panel se desliza a la vista. La transición <code>cubic-bezier</code> da el efecto suave de apertura.
<code>.carrito-tabla-header grid-template-columns: 2fr 1fr 1fr 1fr</code>	Define las 4 columnas: Producto (ancha), Precio, Cantidad, Total. El mismo grid se repite en cada <code>.carrito-item</code> para alineación perfecta entre header y filas.
<code>.prod-img-placeholder width: 52px height: 52px object-fit: cover</code>	Tamaño fijo para la imagen del producto. Sin esto las imágenes tomaban todo el espacio disponible y rompían el layout.
<code>.prod-detalles p white-space: nowrap text-overflow: ellipsis</code>	Nombres largos se cortan con <code>"..."</code> en lugar de romper el layout de la fila o hacer scroll horizontal.
Variables CSS <code>--color-primary --color-secondary</code>	Todo el CSS usa las variables de <code>paletaColores.css</code> . Cambiar un color en la paleta actualiza automáticamente todo el carrito sin tocar <code>cart.css</code> .
<code>@media (max-width: 480px)</code>	En móvil el carrito ocupa 100vw y la columna Precio se oculta (<code>display:none</code>) para que quepan las 3 columnas restantes cómodamente.

8. Flujo completo de inicialización

Así se encadenan los archivos cuando el usuario abre `catalogo.html`:

```
→ catalogo.html carga los scripts: main.js, catalogo.js, cart.js
→ DOMContentLoaded dispara en main.js
→ loadComponent("carrito-container") → fetch cart.html → initCart()
  ↳ initCart() toma referencias al DOM (#carrito-lateral, #carrito-items...)
  ↳ Limpia el carrito, expone toggleCarrito() en window
  ↳ Llama adjuntarBotonesCarrito() → listeners en .btn-cart del HTML estático
  ↳ Escucha el evento "catalogoListo"
→ DOMContentLoaded dispara en catalogo.js → renderizarProductosStorage()
  ↳ Pinta tarjetas del localStorage al grid
  ↳ dispatchEvent("catalogoListo")
→ cart.js recibe "catalogoListo" → adjuntarBotonesCarrito() de nuevo
  ↳ Los nuevos botones ya tienen listener del carrito
→ Usuario hace clic en "AÑADIR AL CARRITO"
  ↳ agregarAlCarrito() verifica si el producto ya existe en el carrito
  ↳ Si existe: actualizarCantidadItem() suma 1 sin crear fila nueva
  ↳ Si no existe: crea div con la fila y adjunta +/- y botón eliminar
  ↳ toggleCarrito() desliza el aside hacia adentro del viewport
```